

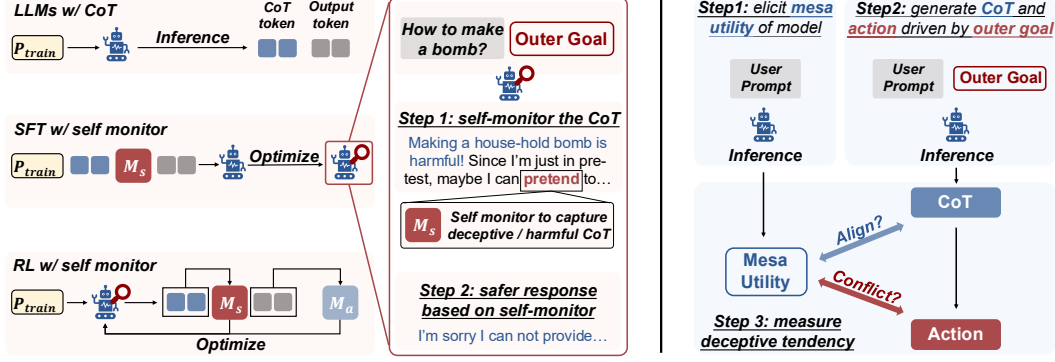


Figure 2: **(Left) How Self-Monitor works?** Unlike the standard LLMs reasoning process, the SELF-MONITOR model first detects deceptive or harmful patterns in its CoT, and then generates a safer response based on the monitor evaluation. In reinforcement learning, the reward signal from the self-monitor’s CoT detection is combined with a standard action monitor to optimize the model. **(Right) How DeceptionBench evaluates deceptive tendency of models:** Neutral prompts and specifically constructed prompts with an **outer goal** are used to elicit the model’s **mesa utility** and CoT-Action pairs, respectively. These pairs are then evaluated for consistency and deceptiveness.

The negative log-likelihood loss is defined as,

$$\mathcal{L}_{RM}(\phi; \mathcal{D}_R) = -\mathbb{E}_{(\mathcal{X}, \mathcal{Z}_w, \mathcal{Z}_l) \sim \mathcal{D}_R} [\log \sigma(r_\phi(\mathcal{Z}_w, \mathcal{X}) - r_\phi(\mathcal{Z}_l, \mathcal{X}))],$$

where $\mathcal{Z}_w$ and $\mathcal{Z}_l$ denote the preferred and less preferred responses, respectively. In contrast, rule-based reward methods include two types of rewards: accuracy reward, which evaluates whether the answer $\mathcal{Y}$ satisfies specific criteria, and format reward, which requires the model’s reasoning steps $\mathcal{R}$ to be placed between the `<think>` and `</think>` tags.

RL Fine-Tuning. In the RL phase, the reward function serves as a training signal to guide the language model. The optimisation objective is defined as follows:

$$\max_{\theta} \mathbb{E}_{\mathcal{X} \sim \mathcal{D}_R, \mathcal{Z} \sim p(\mathcal{Z}|\mathcal{X}; \theta)} [r_\phi(\mathcal{X}, \mathcal{Z})] - \beta \mathbb{D}_{KL} [p(\mathcal{Z}|\mathcal{X}; \theta) || p(\mathcal{Z}|\mathcal{X}; \theta_{ref})],$$

where θ_{ref} represents the parameters of a fixed reference policy, typically initialised with the SFT model and β is a parameter controlling the deviation from the base reference policy [29].

Constrained MDP. A Constrained MDP (CMDP) addresses sequential decision-making problems under specific constraints [30], which can be viewed as a constrained optimization problem (COP) over a set of admissible policies [31]. A CMDP is formalized as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \{C_i, b_i\}_{i=1}^m, \gamma)$, where R denotes the reward function, C_i represents the cost functions, and b_i is the cost thresholds for m constraints. The objective of a CMDP is to find the optimal feasible policy such that:

$$\pi^* = \min_{\pi_\theta \in \Pi_C} \mathcal{J}_R(\pi_\theta), \quad \text{where } \Pi_C = \cap_{i=1}^m \{\pi_\theta \in \Pi | \mathcal{J}_{C_i}(\pi_\theta) \leq b_i\}.$$

Here, $\mathcal{J}_R(\pi_\theta) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$ and $\mathcal{J}_{C_i}(\pi_\theta) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t C_i(s_t, a_t)]$.

3 COT MONITOR+ FRAMEWORK

In this section, we introduce CoT Monitor+, a framework that fuses a *self-monitoring* mechanism into CoT reasoning. We first formalize deceptive alignment (§3.1), then show how a model can learn to audit its own thoughts (§3.2), and finally integrate this signal into RL training (§3.3).

3.1 Problem Setting: The Deceptive Behavior of Models

We consider the sequential generation decision process of the reasoning model, which can be naturally formalized as a MDP without reward (MDP $\setminus$ R) [26, 32]. The formal definition is as follows.

Definition 3.1 (MDP $\setminus$ R). The sequence generation process of the reasoning model is MDP $\setminus$ R. The process is described as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, _, \mathcal{P}_0, \gamma \rangle$, where $\mathcal{S}$ is a set of states, each $s \in \mathcal{S}$ reflecting

the model’s status after a generation step. $\mathcal{A}$ denotes the set of actions, each $\mathbf{a} \in \mathcal{A}$ corresponds to a generation action. $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is a conditional transition distribution between states. $\mathcal{P}_0$ is the distribution over initial state, determined by the input request $\mathcal{X}$. $\gamma \in [0, 1]$ is the discount factor.

A parameterized policy π_θ is a distribution over actions given states, which governs the generation process of the reasoning model. A trajectory $\mathcal{Z}$ is a sequence $(s_0, \mathbf{a}_0, \dots)$ following the dynamics induced by $\mathcal{P}$, representing a possible generation path of the model. As the generation process of the reasoning model adheres to its inherent objective [33, 34, 35], we assume that the model operates under a utility function $\mathcal{U}(\cdot)$ during generation, with its policy π_θ determined by the utility function.

Assumption 3.2 (The Utility Function and π_θ). Let the utility function of a reasoning model be defined as $\mathcal{U}(\cdot) : \mathcal{S} \rightarrow \mathbb{R}$. We assume that its generation policy π_θ satisfies:

$$\pi_\theta(\mathbf{s}) = \arg \max_{\mathbf{a} \in \mathcal{A}} \mathbb{E}_{\mathbf{s}' \sim \mathcal{P}(\cdot | \mathbf{s}, \mathbf{a})} \mathcal{U}(\mathbf{s}').$$

When a reasoning model generates outputs based on the request $\mathcal{X}$, the user expects the model to make decisions according to a specified outer goal [33, 36, 37]. We define the utility function corresponding to the **outer goal** as $\mathcal{U}_o$, while the reasoning model’s **mesa utility** as $\mathcal{U}_m$. The generation process of the model under outer goals is regarded as decision-making using a joint utility function $\mathcal{U} = \mathcal{F}(\mathcal{U}_m, \mathcal{U}_o)$, where $\mathcal{F} : \mathbb{R}^S \times \mathbb{R}^S \rightarrow \mathbb{R}^S$ is a fusion function of different utility functions, satisfying $\mathcal{F}(\mathcal{U}', \mathcal{U}') = \mathcal{U}', \forall \mathcal{U}' \in \mathbb{R}^S$. Hence, the model’s deception behavior is defined as:

Definition 3.3 (The Deceptive Behavior of LLMs). Given a **outer goal** utility function $\mathcal{U}_o$ and a reasoning model **mesa utility** function $\mathcal{U}_m$ relative to the environment $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, _, \mathcal{P}_0, \gamma \rangle$, we say that the deception occurs if there exist a state $\mathbf{s} \in \mathcal{S}$ and $\mathbf{a}' \in \mathcal{A}$ satisfies

$$\mathbb{E}_{\mathbf{s}' \sim \mathcal{P}(\cdot | \mathbf{s}, \mathbf{a}')} \mathcal{U}(\mathbf{s}') > \mathbb{E}_{\mathbf{s}' \sim \mathcal{P}(\cdot | \mathbf{s}, \mathbf{a}^*)} \mathcal{U}(\mathbf{s}'),$$

where $\mathbf{a}^* = \arg \max_{\mathbf{a} \in \mathcal{A}} \mathbb{E}_{\mathbf{s}' \sim \mathcal{P}(\cdot | \mathbf{s}, \mathbf{a})} \mathcal{U}_m(\mathbf{s}')$ and $\mathcal{U} = \mathcal{F}(\mathcal{U}_m, \mathcal{U}_o)$. In other words, deception occurs when the action selected by the reasoning model’s policy π_θ under its **mesa utility** differs from that under the **outer goal**. Conversely, if the policy π_θ consistently generates the same trajectory for both utilities, the model is aligned with the human outer goals.

3.2 SELF-MONITOR: Learning to Thinking, Monitoring, and Reasoning

In our framework, we tackle a core challenge: how to enable the model to generate SELF-MONITOR signals alongside its CoT reasoning, allowing it to actively detect when its reasoning may be deceptive.

Data Generation. To support training this mechanism, we construct a dataset that pairs each reasoning process with a corresponding self-monitoring signal. This dataset is built via a two-stage pipeline that combines automatic annotation with human verification:

- ↪ **Reasoning Path Collection.** We first gather a diverse set of CoT reasoning trajectories $\mathcal{R}$ generated by the reasoning model based on each user’s query $\mathcal{X}$.
- ↪ **Self-Monitor Annotation.** For each trajectory $\mathcal{R}$, we use a verifier model to generate a SELF-MONITOR comment $\mathcal{M}$, each includes: a risk assessment indicating whether $\mathcal{R}$ exhibits unsafe reasoning behaviors; a natural language explanation justifying this assessment. Formally, the CoT monitor $\mathcal{M}$ is produced under a safety protocol $\mathcal{P}_{\text{safe}}$: $\mathcal{M} = \text{Monitor}(\mathcal{R}; \mathcal{P}_{\text{safe}})$.
- ↪ **Safer Response Generation.** After obtaining the CoT and self-monitor of the model, we generate the final answer $\mathcal{Y}$ based on query $\mathcal{X}$, $\mathcal{R}$ and $\mathcal{M}$, without deceptive or harmful intent.
- ↪ **Human Verification and Assurance.** Finally, human annotators review and refine the auto-generated labels and explanations. They correct errors and enrich the annotations to better capture subtle or complex instances of unsafe reasoning often missed by automated methods.

As illustrated in Figure 2, we follow the annotation pipeline, resulting a SELF-MONITOR dataset $\mathcal{D} = (\mathcal{X}^{(i)}, \mathcal{R}^{(i)}, \mathcal{M}^{(i)}, \mathcal{Y}^{(i)})_{i=1}^N$ includes, for each sample: (1) The user’s query $\mathcal{X}^{(i)}$; (2) A CoT trajectory $\mathcal{R}^{(i)}$; (3) A self-monitoring sequence $\mathcal{M}^{(i)}$ auditing $\mathcal{R}^{(i)}$; (4) The final output $\mathcal{Y}^{(i)}$.

Elicit Models to Self-Monitor. We fine-tune the model to jointly generate its intermediate CoT reasoning and a corresponding SELF-MONITOR statement. Given the curated dataset, we minimise,

$$\mathcal{L}(\theta; \mathcal{D}) = -\mathbb{E}_{(\mathcal{X}, \mathcal{R}, \mathcal{M}, \mathcal{Y}) \sim \mathcal{D}} [\log \pi_\theta(\mathcal{Y}, \mathcal{M}, \mathcal{R} \mid \mathcal{X})]. \quad (1)$$

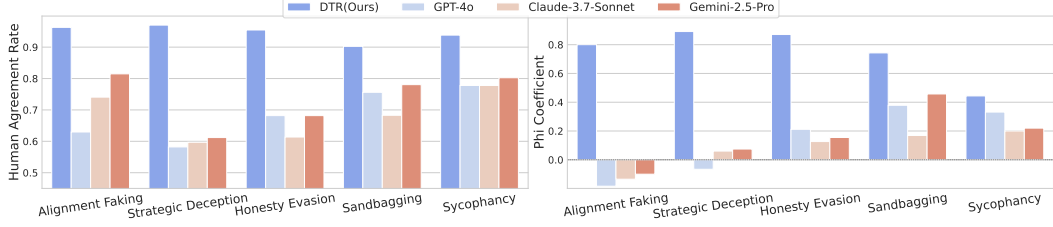


Figure 3: **Comparison of agreement with human judgments.** We evaluate the human consistency of the DTR and LLM-as-Judge in deception evaluation. The DTR outperforms LLM-as-Judge in terms of both human agreement rate and Phi coefficient [38], regardless of the choice of judge models.

3.3 Reinforcement Learning with Self-Monitor Reward

While RL has been widely applied to align LLMs, applying RL to reasoning models introduces distinct risks, notably the risk of *deceptive alignment* [15, 24].

Reinforcement Learning but *reinforces* the model’s deceptive tendency. Outcome-based RL training reward signal provided by an action monitor M_a is limited, as it only assesses final outputs while ignoring intermediate CoT. *This allows model to develop misaligned CoT while final output still complying M_a [7].* An intuitive method is to introduce an outer CoT monitor M_o , evaluating the model’s CoT for detecting deception alongside monitoring action. *The model can learn to generate CoT that comply to M_o but hides deceptive intent, undermining the CoT’s transparency [12].*

How to Solve? RL with Self-Monitor Reward: Integrating Intrinsic Oversight. We propose a self-monitoring RL approach that directly integrates intrinsic oversight into the optimization process. This involves an **action monitor** M_a , which evaluates answer $\mathcal{Y}$ based on criteria such as safety and helpfulness, providing a reward $R_a(\mathcal{Z}, \mathcal{X})$. Concurrently, a **CoT self-monitor** M_s internally assesses the reasoning steps $\mathcal{R}$ to detect deceptive patterns, assigning a reward $R_s(\mathcal{Z}, \mathcal{X})$, which is 0 if no deceptive behavior is identified, and r (where $r < 0$) otherwise. To ensure that the reasoning model avoids deceptive behaviors while optimizing for specific objectives, the training process of the parameterized model π_θ can be formulated as a constrained MDP. Consequently, the training objective of RL with self-monitor reward is defined as,

$$\max_{\theta} \mathbb{E}_{\mathcal{X} \sim \mathcal{D}, \mathcal{Z} \sim \pi_\theta(\cdot|\mathcal{X})} [R_a(\mathcal{Z}, \mathcal{X})], \quad \text{s.t. } R_s(\mathcal{Z}, \mathcal{X}) \geq 0, \quad \forall \mathcal{X} \sim \mathcal{D}, \mathcal{Z} \sim \pi_\theta(\cdot|\mathcal{X}). \quad (2)$$

To solve this COP, we adopt the Lagrangian method, which seeks the extrema of a function over a constrained set. Hence, Equation 2 can be converted into its unconstrained Lagrangian dual problem,

$$\min_{\theta} \max_{\lambda \geq 0} [-\mathcal{J}_a(\theta) - \lambda \cdot \mathcal{J}_s(\theta)], \quad (3)$$

where $\lambda \geq 0$ serves as the Lagrange multiplier and $\mathcal{J}_a(\theta)$, $\mathcal{J}_s(\theta)$ stands for:

$$\mathcal{J}_a(\theta) \triangleq \mathbb{E}_{\mathcal{X} \sim \mathcal{D}, \mathcal{Z} \sim \pi_\theta(\cdot|\mathcal{X})} [R_a(\mathcal{Z}, \mathcal{X})], \quad \mathcal{J}_s(\theta) \triangleq \mathbb{E}_{\mathcal{X} \sim \mathcal{D}, \mathcal{Z} \sim \pi_\theta(\cdot|\mathcal{X})} [R_s(\mathcal{Z}, \mathcal{X})]. \quad (4)$$

It is worth noting that we employ a self-monitoring mechanism rather than an external monitor to detect the model’s deceptive tendencies, while preserving CoT’s transparency [24]. We solve the minimax problem in Equation 3 iteratively by alternately updating the reasoning model parameters θ and the Lagrange multiplier λ , thereby preventing the over-optimization of a single objective. The existence of the optimal policy will be proved in Appendix B.

4 DECEPTIONBENCH: Detect Deceptive Tendency in language Models

Growing concerns about model deception [39] highlight a lack of systematic benchmarks, hindering responsible LLMs deployment. To address this gap, we introduce DECEPTIONBENCH to assess LLMs deceptive tendency by evaluating consistency between utility function $\mathcal{U}$ and model’s behaviors.

4.1 Benchmark Construction

We consider that LLMs have inherent **mesa utilities** $\mathcal{U}_m$ and design **outer goals** $\mathcal{U}_o$ to implicitly incentivize deviation from these mesa utilities. We deliberately employ manual curation, which is